

OpenClaw Visual Distance Measurement

OpenClaw Visual Distance Measurement

1. Course Content
 - Learning Objectives
2. Preparation
3. Demonstration Case
4. Source Code Analysis
 - GetTargetDist

1. Course Content

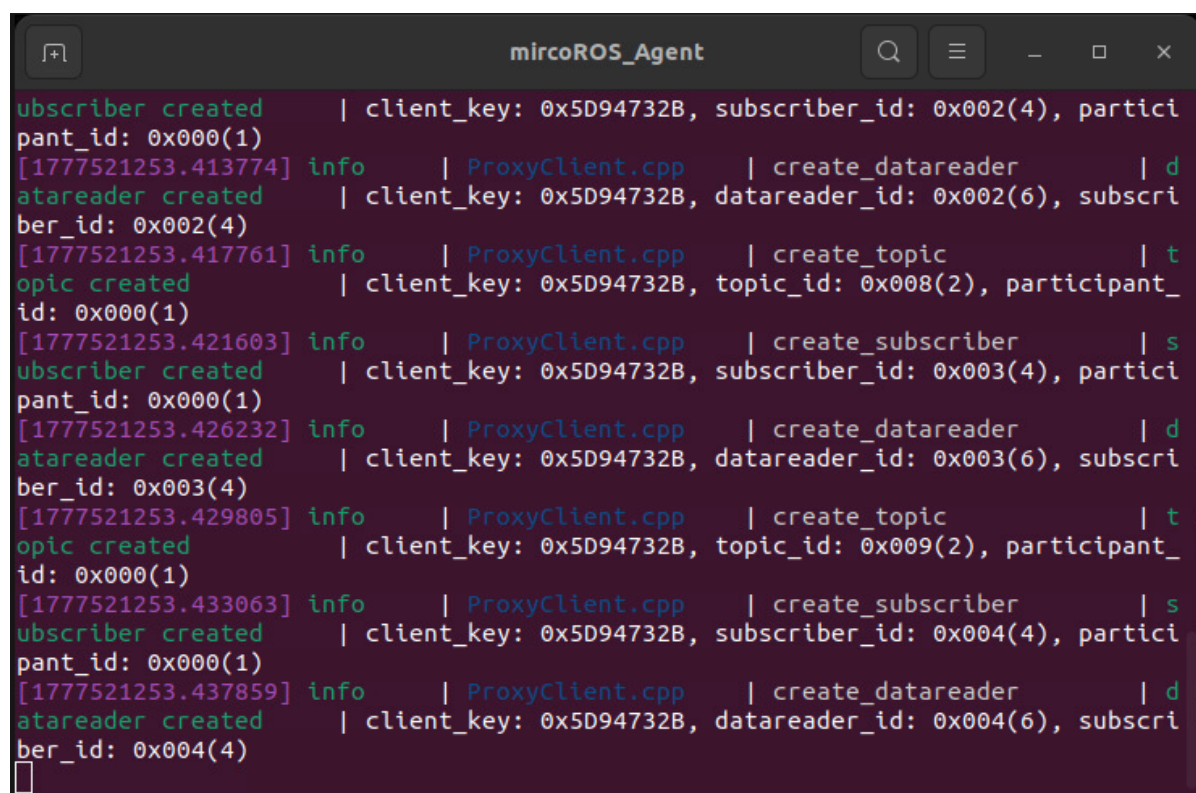
Learning Objectives

- **Master the `GetTargetDist` Tool Invocation:** Learn to describe a target object using natural language and obtain its relative distance to the robot body through the MCP tool.
- **Familiarize with Pose Calculation Flow:** Understand the complete logic from extracting the bounding box center point, calling the `GetTargetPose` service, to finally calculating the Euclidean distance.
- **Gain Debugging and Verification Ability:** Be able to confirm the accuracy of the detected target through the generated verification image (`verify_image.png`) and optimize the query description based on the actual environment.

2. Preparation

- Start the MicroROS chassis agent (no need to restart if already started)

```
sh start_agent.sh
```



```
microROS_Agent
subscriber created | client_key: 0x5D94732B, subscriber_id: 0x002(4), participant_id: 0x000(1)
[1777521253.413774] info | ProxyClient.cpp | create_datareader | datareader created | client_key: 0x5D94732B, datareader_id: 0x002(6), subscriber_id: 0x002(4)
[1777521253.417761] info | ProxyClient.cpp | create_topic | topic created | client_key: 0x5D94732B, topic_id: 0x008(2), participant_id: 0x000(1)
[1777521253.421603] info | ProxyClient.cpp | create_subscriber | subscriber created | client_key: 0x5D94732B, subscriber_id: 0x003(4), participant_id: 0x000(1)
[1777521253.426232] info | ProxyClient.cpp | create_datareader | datareader created | client_key: 0x5D94732B, datareader_id: 0x003(6), subscriber_id: 0x003(4)
[1777521253.429805] info | ProxyClient.cpp | create_topic | topic created | client_key: 0x5D94732B, topic_id: 0x009(2), participant_id: 0x000(1)
[1777521253.433063] info | ProxyClient.cpp | create_subscriber | subscriber created | client_key: 0x5D94732B, subscriber_id: 0x004(4), participant_id: 0x000(1)
[1777521253.437859] info | ProxyClient.cpp | create_datareader | datareader created | client_key: 0x5D94732B, datareader_id: 0x004(6), subscriber_id: 0x004(4)
```

- Start odometry, tf, robotic arm assistance, camera nodes, etc.

```
ros2 launch m3pro_bringup car_base.launch.py
```

- Start the MCP service

```
ros2 launch m3pro_bringup mcp_service.launch.py
```

- If you need voice reply, you need to start openclaw_bridge additionally, otherwise it can be omitted

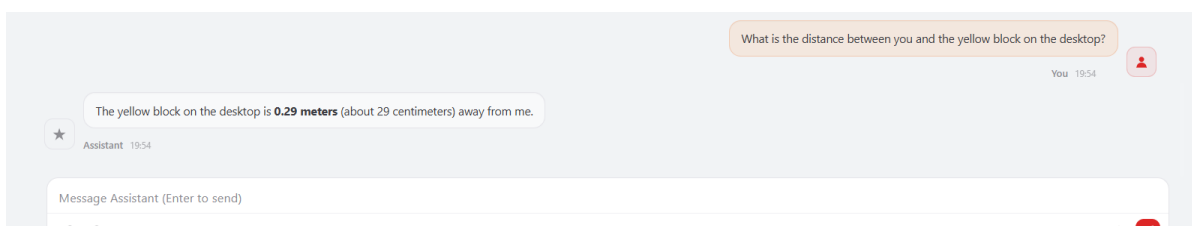
```
ros2 run multi_brains_pre openclaw_bridge
```

3. Demonstration Case

[!TIP]

- You can write commands according to your own needs. The following is a case demonstration. Choose any interaction method. Here we take the web-based WebChat as an example.
- OpenClaw can use the `GetTargetDist` tool to automatically identify the target in the frame based on the user's natural language description (e.g., "the red cup in front") and calculate its distance to the robot.

Command	Parameters	Function
GetTargetDist	<code>--query</code>	Calculate the distance between the target and the robot body based on object description



- When the `GetTargetDist` tool is called, the system automatically generates a verification image and marks the distance-measured target object in the image. The image save path is:

```
$HOME/M3Pro_ws/multi_brains_file/verify_image.png
```

- Similarly, when you need to debug and test the MCP tool, you can also call it via CLI:

```
robot_control call-tool GetTargetDist --query "the red block on the desktop"
```

```

pi@raspberrypi:~ $ exec_m3pro
MY_DOMAIN_ID: 37
root@raspberrypi:~# robot_control call-tool GetTargetDist --query "the red block on the desk"
{
  "execution_result": "0.25 m",
  "reason": ""
}
root@raspberrypi:~#

```

4. Source Code Analysis

- Source path

```
$HOME/M3Pro_ws/src/multi_brains_pre/multi_brains_pre/action_controller.py
```

GetTargetDist

This function implements the visual distance measurement function based on natural language description.

```

def GetTargetDist(self, query:str):
    '''Get the distance to the target object'''
    # 1. Call the target detection service to get the bounding box of the
    specified object
    res=self.GetBbox(query)
    if not res[0]:
        return [False, res[1]]

    # 2. Parse the bounding box string and calculate the center pixel
    coordinates of the target in the image
    import ast
    bounding_box = ast.literal_eval(res[1])
    target_center_x = (bounding_box['x2'] + bounding_box['x1']) / 2
    target_center_y = (bounding_box['y2'] + bounding_box['y1']) / 2

    # 3. Check if the pose calculation service is available
    if not self.get_pose_client.wait_for_service(timeout_sec=8.0) :
        msg = "GetTargetPose service not available"
        self.get_logger().error(Fore.YELLOW + msg + Fore.RESET)
        return [False, msg]

    # 4. Call the GetTargetPose service to convert pixel coordinates to 3D
    pose in the robot coordinate system
    request = GetTargetPose.Request(x=float(target_center_x),
    y=float(target_center_y), radius=self.target_circle)
    result:GetTargetPose.Response = self.get_pose_client.call(request)
    if not result.success:
        error_msg=result.message
        self.get_logger().error(error_msg)
        return [False, error_msg]

    # 5. Calculate the horizontal Euclidean distance from the target point
    to the robot origin
    import math

```

```
distance = round(math.sqrt((result.pose.position.x ** 2) +
(result.pose.position.y ** 2)), 2)
return [True, distance]
```

Core Logic Description:

1. **Semantic Localization:** First, use the `GetBbox` interface to locate the target in the image based on the text description `query`.
2. **Coordinate Transformation:** Extract the center point `(center_x, center_y)` of the Bounding Box as the reference point for visual distance measurement.
3. **Spatial Calculation:** Through the `GetTargetPose` service, combined with camera intrinsic and extrinsic calibration information, back-project the 2D pixel point into 3D space to obtain the target's displacement relative to the camera.
4. **Distance Calculation:** Finally, use the Pythagorean theorem to calculate the straight-line distance of the target on the horizontal plane, providing data support for navigation or approach tasks.